The CSP Dichotomy Theorem

A formalization blueprint

Abstract

This is the formalization companion to *The CSP Dichotomy Theorem*, a corrected exposition of Zhuk's simplified proof [14]. It carries the material that is about a Lean 4 development rather than about the mathematics: which of the five statements a formalization would be proving and what each costs (§1–§2), the representation decisions and their reasons (§3.3), the layers Mathlib does not supply (§3.4), the order in which the theorems should be attacked (§3.5), and what governs the schedule (§3.6).

References of the form `lem:...` are labels of statements in the proof paper, which is compiled separately; the two documents share no cross-reference machinery on purpose, so that neither can be read as evidence for the other. Claims here about the state of a repository are claims about a repository, and should be checked against a commit hash rather than believed.

Contents

1 What a formalization can claim

The dichotomy theorem, as stated in the proof paper, is conditional on a model of computation: "solvable in polynomial time" quantifies over an encoding of instances as strings, a machine, and a polynomial bound on its step count, and "NP-complete" additionally quantifies over all problems in NP and over polynomial-time many-one reductions. Mathlib supplies Turing machines (`Mathlib/Computability/TuringMachine/`) and a notion of computability, but it has *no* complexity class, no polynomial-time predicate, and no notion of polynomial-time reduction. So a formalization must say which of the statements it is proving, and the proof paper's five layers are the answer.

1.1 Where the work actually is

An earlier draft of this section concluded from that gap that the complexity-theoretic half should be deferred because building it would dominate the project. That inference was wrong, and it is worth recording why, because the corrected version is what organises the rest of this document.

The complexity vocabulary itself — languages, a cost model, P, NP by verifiers, $\leq_p$, NP-hardness, NP-completeness, and a generic NP-hard problem — has since been built, in about 850 lines. That is consistent with the two existing Cook–Levin formalizations, whose class layers are 702 lines (Isabelle) and 992 lines (Coq) against totals of 56 514 and $\approx$ 16 500. Defining the classes is cheap in every development that has done it.

What is expensive is not complexity theory. It is any statement that requires one to *exhibit a program*. In a formalization, "f is computable in polynomial time" is not a property one observes of a mathematical function; it is an existential over programs, discharged only by writing one and proving a bound on its step count. The dividing line runs along that axis, and it cuts across the algebra/complexity split rather than agreeing with it:

Target	Exhibit a program?	Cost
(T0) the algebraic core	no	the mathematics
(T1) Solve is correct	no (but see Formalization note 1.1)	moderate
(T2) Solve is polynomial	**yes — the largest one here**	**large**
(H0) Γ pp-interprets Nae	no	the mathematics
(H1) pp-interpretation $\Rightarrow \leq_p$	yes, a syntactic substitution	small

The blueprint is written for (T0), (T1) and (H0). (H1) is a small addition on top; (T2) is neither small nor mathematics, but it is not *unrelated* to the dichotomy either — it is the assertion that the algorithm one has just proved correct is also fast, and it is the second-largest component of the project.

Formalization note 1.1 ((T1) is vacuous unless Solve computes)**.** "Solve written as a total function" hides a requirement that no cost model would catch. If Solve is defined using classical choice — for instance by deciding "does D_x have a nonempty proper binary absorbing subuniverse?" with `Classical.dec` — then $\textsc{Solve}(\mathcal{I}) = \texttt{true} \iff \mathcal{I}$ has a solution is a true theorem that says nothing algorithmic, because Solve does not reduce. To mean what it appears to mean, (T1) needs every predicate the algorithm branches on to carry a genuine decision procedure.

That is a real obligation with a locatable cost. Absorption is defined by an existential over *terms* — an infinite type — so "C is a binary absorbing subuniverse of B" is not decidable on its face. It becomes decidable through a bound on the arity of a sufficient witness, which is a theorem and not a definition. For the types actually used here the bound is cheap and should be taken that way rather than from a general absorption-arity theorem: ba has

a binary witness by definition, and central absorption has a ternary witness by the proof paper, `lem:central-ternary`. Only finitely many binary and ternary operations then need be checked, together with finitely many congruences and subsets. The same remark applies to the searches inside SolveLinear.

Formalization note 1.2 ((T2) is not a wrapper)**.** Two things make (T2) much larger than the phrase "complexity wrapper" suggests.

The program is enormous. (T2) requires Solve — ForceConsistency, the search for a strong subuniverse, SolveLinear, and the mutual recursion between them — to be expressed in a cost model, with a proved polynomial bound. Every existing measurement says this kind of work dominates: it is the 50 000 lines of Turing-machine engineering in the Isabelle Cook–Levin development, against 702 for the classes themselves.

The statement is non-uniform. Because Γ is fixed, the polynomial may depend on Γ, and the searches over subsets of A, over congruences on A, and over term operations of bounded arity are all constant-time. But "constant" here means the search is compiled away into a program whose *size* depends on $|A|$. So Solve is not one program: it is a family $\textsc{Solve}_\Gamma$ with a polynomial p_Γ for each Γ, and (T2) must be stated in that form. Attempting a bound uniform in Γ would be proving something else, and something false.

Formalization note 1.3 (From pseudocode to a Lean function)**.** (a) *Termination is not structural.* Solve recurses on strictly smaller domains, and SolveLinear loops on a strictly decreasing dimension. Neither is a structural recursion; both need an explicit well-founded measure, and the two interleave — SolveLinear calls Solve on a smaller domain, which calls SolveLinear again. The measure is lexicographic: $(\sum_x |D_x|,\ \dim \phi)$.

(b) *Several steps quantify over objects that are only finitely many by accident.* "If some D_x has a strong subset B" quantifies over subsets of D_x, over congruences on D_x, and — through absorption — over *term operations*, of which there are infinitely many. See Formalization note 1.1 for the route to decidability that does not depend on an uncited general arity bound.

(c) *Polynomial time is a different subject.* The bound in [13] counts calls and bounds the recursion depth by $|A|$; making that formal needs a cost model. See §2.

2 The complexity layer

Targets (T2) and (H1) are stated here and not proved. The point of stating them is to be precise about what is *not* being claimed.

Definition 2.1 (What a cost model would have to supply)**.** To state (T2) one needs: an encoding of instances of CSP(Γ) as strings; a machine model with a step count; a predicate "f is computed in time bounded by a polynomial in the input length"; and closure of that predicate under composition. To state (H1) one needs, in addition, a polynomial-time many-one reduction relation $\leq_p$ and the class NP.

Mathlib supplies the machine (`Turing.TM2`), an encoding (`Computability.Encoding`), and a time-bounded computation predicate (`Turing.TM2ComputableInPolyTime`). It does not supply a complexity *class*, P, NP, NP-completeness, $\leq_p$, or Sat; and the composition lemma `TM2ComputableInPolyTime.comp` is an open `proof_wanted` in Mathlib itself. Moreover `TM2ComputableInPolyTime` applies to total functions $\alpha \to \beta$, never to languages, so even the existing predicate would need reshaping.

Remark 2.2 (This layer now exists, and it was cheap)**.** An earlier draft of this remark said that building the complexity layer was "a substantial project — comparable in size to the

algebraic core" and that attempting it as part of this project "would be a scoping error". That was wrong, and it has been tested rather than argued: the vocabulary — words, a cost model, P, NP by verifiers, $\leq_p$, NP-hardness, NP-completeness, and a generic NP-hard problem — is about 850 lines of Lean, `sorry`-free.

So the reason to keep (T2) out of scope is *not* that complexity theory is unavailable. It is Formalization note 1.2: (T2) requires exhibiting Solve as a program with a proved step bound, and that is the largest program-construction task in the project. (H1), by contrast, needs only a syntactic substitution and should be built.

Remark 2.3 (What the exercise cost, and what it caught)**.** Two defects surfaced in those 850 lines, both in *statements* rather than proofs, and neither of a kind a type-checker reports. The generic problem was first defined with a single budget bounding both the certificate length and the running time, which makes the theorem false — the two directions pull the bound opposite ways. And the language was first given only the tests "register is empty" and "register's first bit is 1", with which no loop that consumes its input can be written at all; that was found by trying to write the first program.

The ratio is worth noting for the rest of this project. Two statement-level errors in 850 lines, against fourteen found in Zhuk's fifty pages by readers who did not formalize. The defect list of the audit document should be expected to grow under formalization, not to be complete.

Remark 2.4 (NP membership)**.** The proof paper's hard half concludes NP-hardness. Recovering NP-completeness needs the standard theorem that every fixed finite-template CSP lies in NP under the chosen encoding: a solution is a certificate of size linear in the number of variables, and checking it means evaluating each constraint, which for fixed Γ is a table lookup. It is small, it needs the cost model, and it belongs with (H1).

3 The Lean development

3.1 What is reused

The predecessor development `ZhukLean` is a complete, `sorry`-free Lean 4 formalization of Zhuk's centre theorem, built on `Mathlib.ModelTheory`. It is consumed here as a `lake` dependency, not copied. It supplies:

- products of structures (`piStructure`, `prodStructure`, coordinatewise realization, reindexing and evaluation homomorphisms) — the one part of the universal-algebra vocabulary Mathlib lacks;
- absorption in the tuple form (`Witnesses`, `Absorbs`, `BinAbsorbs`), Taylor terms (`IsTaylorOn`), and central absorption (`CentrallyAbsorbs`);
- the relational description of absorption by essential relations (Barto–Kazda), which is what converts absorption at some arity into absorption at arity 3;
- Zhuk's centre theorem itself (`zhuk_center`) and the ternary collapse (`exists_ternary_witnesses`) — which is precisely the proof paper, `lem:central-ternary`, cited without proof in [14].

Those also discharge, in advance, every ingredient of [15, Lem. 6.11], which the proof paper's stable-intersection theorem needs in its type-C case: essentiality, the Barto–Kazda criterion, and "central implies ternary absorbing".

3.2 What is built here

Module	Contents	Status
`CSP/Product`	products of structures	complete
`CSP/Congruence`	`Congruence L M`, order, `sInf`, quotient	complete
`CSP/Irreducible`	irreducibility, existence and uniqueness of σ^*	complete
`CSP/Bridge`	bridges, collapse, composition, linear/PC	2 imports open
`CSP/Types`	the six types, multi-types, $\lhd\lhd\lhd$	complete
`CSP/Instance`	instances, reductions, consistency, cruciality	complete
`CSP/Main`	the main statements	targets

Everything below `CSP/Main` is `sorry`-free. The two open items in `CSP/Bridge` are the proof paper, `lem:bridge-comp` and its collapse identity, both imported from [13]. These are claims about a repository, not about the mathematics; they should be read against a commit hash, and they are deliberately kept out of the proof paper for that reason.

3.3 Design decisions

Build on `Mathlib.ModelTheory`, keep the language generic. A bespoke "finite algebra with one WNU operation" type would bundle the standing hypotheses as fields and avoid threading them, which is a real cost in the predecessor development. But it would discard `Substructures.lean` — 985 lines of exactly the Sg API needed, including `mem_closure_iff_exists_term` with the variable type equal to the generating set, an interface no hand-rolled clone matches — and it would discard the predecessor's 1600 lines wholesale. Congruences and quotients are also *cheaper* in `ModelTheory`, not dearer. Instantiating to $\mathcal{V}_n$ is a thin layer on top. Carry the standing hypotheses as typeclass instances on the structure rather than as ambient hypotheses, so that the audit of which result consumes which hypothesis is mechanical.

Terms and term operations are different types. [14] writes $t \in \mathrm{Clo}(\mathbf{A})$ and treats t as both a syntactic term and the operation it induces. Every statement of the proof paper

that quantifies over $\mathrm{Clo}(\mathbf{A})$ is about *operations*, so the syntactic layer may be quotiented away — but it cannot be skipped, because Sg is characterised through terms and because the arity bookkeeping in the absorption arguments is syntactic.

$\lhd\lhd\lhd$ is a witnessed chain, not `Relation.ReflTransGen`. An earlier draft of this section said the opposite, and it was wrong: the proof paper's "the congruences coming from $C \lhd\lhd\lhd^A B$" is a function of the chain and not of the pair, and two arguments turn on *which* chain is chosen. So the primary notion is an inductive family carrying, at each step, the intermediate set, its type, and its witnessing congruence, with extension by one step as an operation on that data; `Relation.ReflTransGen` is its propositional truncation and may be used only where no congruence is named. Both are needed: the truncation supplies the induction principle most consumers want.

Dotted relations are defined as disjunctions. `DotLll C B := C = ∅ ∨ Lll C B`, so the case split is forced at every use site rather than resolved by the reader. Conflating $\lhd\lhd\lhd$ with $\dot\lhd\dot\lhd\dot\lhd$ is the commonest way to misread the type theory of the proof paper's §3.

Absorption is stated over coordinatewise-constrained tuples. That is: for every $\bar a \in A^k$ with $a_j \in B$ for all $j \neq i$, one has $t(\bar a) \in B$. *Not* as "for all $b_1, \dots, b_k \in B$ and $a \in A$, $t(b_1, \dots, a, \dots, b_k) \in B$". The two agree, but the second phrasing invites the variant in which b_i is quantified and then discarded, which is vacuously true when $B = \varnothing$ and at $k = 1$ where the intended content is not vacuous. This was an actual defect in the predecessor blueprint.

The bridge criterion is the definition of linearity. Zhuk defines a linear congruence by a per-block isomorphism to $\mathbb{Z}_p^m$ and derives the bridge criterion. We reverse this: the criterion is first-order over finite data and is what every consumer uses; the block description becomes a structure theorem. See the proof paper, `haz:linear-def`.

σ^* is data attached to irreducibility. `Irreducible.exists_isCover` produces it; there is no standalone function, because there is no cover without irreducibility. A pleasant by-product: with the definition taken literally, irreducibility *implies* $\sigma \neq A^2$, since the empty family intersects to the universe.

Con_1 gets a neutral name until it is a congruence. The raw relation is always reflexive and symmetric but not transitive; the notation invites the reader to assume congruence-hood. Reserve Con_1 for the case where subdirectness and rectangularity make it one. Note also that it depends on the *position* in the scope, not on the variable, so it is ill-defined if a variable occurs twice.

Instances are lists; scopes are tuples; the variable type is a parameter. Proofs delete a constraint and reason about $\mathcal{I} \setminus \{C\}$, and the same relation may occur twice on the same tuple inside an expanded covering, so `List` with membership by index is the safe choice. Repeated variables in a scope are allowed and occur.

Coverings are built by coproduct of variable types, not by renaming. [14] writes "we rename the variables so that the only common variable of Υ_x and $\mathcal{J}$ is x". With a concrete variable type this means a fresh-name supply, a renaming operation, and a lemma that renaming preserves each of 1-consistency, cycle-consistency, cruciality, and being a covering — eight lemmas the paper does not state. Making the variable type a parameter and gluing over $V_1 \oplus V_2$ quotiented by the shared variable turns all eight into transport along an equivalence. This is the single most important representation

decision below the algebra, and it should be made before anything about coverings is written.

Nothing is stubbed with `sorry` at the level of a definition. `IsConnectedInstance` and `IsExpandedCovering` are *absent* rather than defined as `sorry`, because a `def : Prop := sorry` is a junk constant that downstream proofs can silently lean on. Absence forces the design decision; a stub hides it.

3.4 What Mathlib is missing

Layer	Status
Products of first-order structures	Absent from Mathlib; supplied by `ZhukLean` (§3.1).
Congruences of a structure, quotients	`ModelTheory` has substructures but not congruences; built here in `CSP/Congruence`.
Absorption, central subuniverses	Absent; supplied by `ZhukLean`.
Mixed-prime "dimension"	A subgroup of $\prod_i \mathbb{Z}_{q_i}$ with distinct q_i is not a vector space. `ZMod` and `Module (ZMod p)` exist; the primary decomposition into $\mathbb{F}_q$-vector spaces and the codimension-one characterisation must be built. Needed by the proof paper's codimension-one theorem.
The Pol–Inv Galois connection	Absent. Roughly four printed pages, of independent value, and the single largest item in the (H0) budget.
Complexity classes, $\leq_p$, NP	Absent; see §2. Built separately in ≈ 850 lines.
Polynomial completeness, Istinger–Kaiser	Absent. Needed only for the proof paper, `lem:pc-on-top`, which nothing else consumes; a staged development should treat "PC congruence" as "irreducible and not linear" throughout and prove that lemma last or not at all.

3.5 Order of attack

1. the proof paper, `lem:ba-center-implies`. Small, self-contained, used everywhere, and adjacent to what is already formalized. Its statement needs both hypotheses the source drops — subdirectness, and a common binary term — and the common term should be carried in the type, as [15] writes $\leq_{\mathrm{BA}(t)}$.
2. The mixed-prime linear algebra of §3.4. Independent of everything else; can be done in parallel.
3. the proof paper, `lem:bridge-comp`. Twelve source lines of relational algebra; closes the two open items in `CSP/Bridge`.
4. The coproduct design for coverings, then the basic properties of expanded coverings.
5. the proof paper, `thm:stable-intersection`. The one hard statement in the interface, and the sole source of bridges.
6. the proof paper, `lem:bridge-from-instance`. Eight hypotheses, ten pages.
7. the proof paper, `thm:main-inductive`. Last.

3.6 Scope, and what governs the schedule

The calibration point is that the predecessor formalized about 3.5 printed pages in about 1670 lines of Lean — roughly 480 lines per page. Excluding §2 and [14]'s §4, which is an independent second result and not needed for the dichotomy, the transitive closure of what must be proved is on the order of 100–130 printed pages, so 35 000–60 000 lines of Lean.

Line count is not the binding constraint, and it should not be converted into person-time here: any such figure would be a claim about a development process rather than about this mathematics, and it is not evidence a reader of this document can check.

What binds is the *critical path*: the depth of the serial chain, and the number of places where the source is wrong and new mathematics is needed. The serial chain is

$$\texttt{thm:stable-intersection} \longrightarrow \texttt{lem:bridge-from-instance} \longrightarrow \texttt{thm:main-inductive}$$

each a single large proof that cannot be split across workers — stable intersection is the sole source of bridges, bridge-from-instance is the only consumer that turns them into instance-level structure, and the main induction consumes that.

The new mathematics is a much shorter list than an earlier draft of this section claimed, and the items on it are not the ones it named. Of the audit document's register, thirteen defects are repaired with no new mathematics. What remains open, and gates the formalization of §5 of [14], is:

- the proof paper, `lem:maximal-mult` — the maximal multi-type extension. Its Case 1 is unproved and the published repair is invalid; see the audit document. Nothing downstream of it should be attempted first.
- The rectangularity normalisation "compose the bridge with itself enough times", on which the proof paper, `lem:no-cross-bridge` and the proof paper, `lem:bridge-to-pc` rest. The reflexivisation that used to stand beside it on this list is *not* open: it is an observation about a hypothesis every consumer already carries, and it is now the proof paper, `lem:bridge-reflexive`.
- the proof paper, `lem:self-intersection-pc`, now the sole outline under the proof paper, `lem:intersection-good`, which is otherwise written out in full; and the proof paper, `lem:multiply-all-linear`, which is not yet a proof.

Those are not throughput-limited. Schedule around them, not around the line count.

References

[1] A. Bulatov, P. Jeavons, A. Krokhin, *Classifying the complexity of constraints using finite algebras*, SIAM J. Comput. **34** (2005), 720–742.

[2] Z. Brady, *Notes on CSPs and polymorphisms*, arXiv:2210.07383. Cited by section title and numbered statement; the sections used here are "Cores and Idempotent Reducts" and §3.11, §3.15.

[3] L. Barto, M. Kozik, *Absorbing subalgebras, cyclic terms, and the constraint satisfaction problem*, Log. Methods Comput. Sci. **8** (2012), 1:07.

[4] L. Barto, A. Kazda, *Deciding absorption*, Internat. J. Algebra Comput. **26** (2016), 1033–1060.

[5] C. Bergman, *Universal algebra: fundamentals and selected topics*, CRC Press, 2011.

[6] A. Bulatov, *A dichotomy theorem for nonuniform CSPs*, arXiv:1703.03021; FOCS 2017.

[7] T. Feder, M. Y. Vardi, *The computational structure of monotone monadic SNP and constraint satisfaction*, SIAM J. Comput. **28** (1999), 57–104.

[8] D. Hobby, R. McKenzie, *The structure of finite algebras*, Contemp. Math. 76, Amer. Math. Soc., 1988.

[9] M. Istinger, H. K. Kaiser, *A characterization of polynomially complete algebras*, J. Algebra **56** (1979), 103–110.

[10] M. Maróti, R. McKenzie, *Existence theorems for weakly symmetric operations*, Algebra Universalis **59** (2008), 463–489.

[11] L. Barto, Z. Brady, A. Bulatov, M. Kozik, D. Zhuk, *Minimal Taylor algebras as a common framework for the three algebraic approaches to the CSP*, LICS 2021.

[12] R. Willard, *Similarity, critical relations, and Zhuk's bridges*, slides, AAA98 — 98th Workshop on General Algebra, Dresden, 2019. `http://www.math.uwaterloo.ca/~rdwillar/documents/Slides/willard-aaa98-handout.pdf`

[13] D. Zhuk, *A proof of the CSP dichotomy conjecture*, J. ACM **67** (2020), no. 5, 1–78; arXiv:1704.01914.

[14] D. Zhuk, *A simplified proof of the CSP dichotomy conjecture and XY-symmetric operations*, arXiv:2404.01080v2.

[15] D. Zhuk, *Strong subalgebras and the constraint satisfaction problem*, J. Mult.-Valued Logic Soft Comput. **36** (2021); arXiv:2005.00593.